

GST BILLING & INVENTORY MANAGEMENT SYSTEM

SCIENCE HUB

The Complete Technical & Business Documentation
("The Science Hub Bible")

Prepared for engineering, operations, and business stakeholders

Fifth Edition | Updated 2026-09-17 | Verified directly against the live source code

d:\nextApps\science-hub — Next.js 16 • Prisma • PostgreSQL (Neon) • Vercel

Table of Contents

1. Executive Summary

2. High-Level System Overview

3. Complete Technology Stack

4. Folder Structure

5. Application Flow

6. User Roles & Permissions

7. Authentication & Sessions

8. Database (Full ER Reference)

9. Complete Module Documentation

10. Invoice Flow — Step by Step

11. Stock Flow & the Movement Ledger

12. GST Logic

13. API Documentation (Full Reference)

14. Frontend Architecture

15. Backend Architecture

16. Data Flow

17. PDF Generation

18. Email System

19. Security

20. Error Handling

21. Deployment

22. Third-Party Services

23. Environment Variables

24. Project Dependencies

25. Business Workflow (A Day in the Life)

26. Non-Technical Explanation

27. Developer Guide

28. Future Improvements

How this document was built

Every claim in this document was verified by reading the actual source code at `d:\nextApps\science-hub` — not guessed, and not copied blindly from the project's own internal notes (`CLAUDE.md`, `DEVELOPMENT_LOG.md`). This is the same methodology every prior edition established, carried forward unchanged: internal notes were used as a map of where to look, then every load-bearing claim was spot-verified directly against the relevant source file before being written down here. This edition was produced by nine parallel research passes, each independently verifying its assigned chapters against the live codebase (reading route handlers, library modules, the Prisma schema, and `git log` / `git show` history directly) rather than trusting the prior pass's conclusions — several genuine inconsistencies between `CLAUDE.md`'s own notes and the actual current code were found this way and are called out explicitly below and throughout, in the same "Update" callout style prior editions used for corrections.

What's new in this Fifth Edition (since the Fourth Edition, 2026-08-31)

Roughly two and a half weeks of continuous, dated work (2026-09-02 → 2026-09-17) is covered in this edition — the longest and busiest window any edition of this Bible has documented. In rough chronological order:

- **GSTR-1 Offline Tool CSV export** ("Tier 1, no GSP"), merged 2026-09-02 — a ZIP of section-wise CSVs matching the free government GST Returns Offline Tool's own column format, alongside the existing ZIP/ `.xlsx` GST filing package.
- **A second full 10-specialist audit pass** (2026-09-05) closing 38 findings across security, data integrity, performance, and accessibility — including a real TOCTOU fix making the customer credit-limit check run inside the same transaction as an invoice write, not before it.
- **Product pricing consistency** (2026-09-06) — List Price/Discount % now drive Purchase Price end-to-end on both Product and Purchase Bill, GST rate freed from a fixed dropdown to free-text 0–100, and a shared `QuickAddProductModal` replaced two near-duplicate popups.
- **Shared payment-method list, IMPS, and zero-padded document numbers** (2026-09-06).
- **Payment edit/delete** for both sales and purchase (2026-09-07) — the first way to correct a mistake in an already-recorded payment, anywhere in the app.
- **Purchase-bill attachments moved to a private Vercel Blob store** (2026-09-08) — closing a real confidentiality gap where an uploaded vendor bill's raw file URL had been publicly readable since the feature was introduced.
- **A permanent close-out of the recurring IST/UTC date-boundary bug class** (2026-09-09), this time with an automated pre-commit guard rather than another manual fix, after the same bug shape had already resurfaced three times.
- **A PDF-pipeline polish pass** (2026-09-09) fixing a genuinely subtle multi-page header-repeat bug, plus a cache-invalidation audit closing a whole family of stale-cache bugs across the app.
- **Next-document-number preview badges** (2026-09-09), an app-wide **loading-skeleton fidelity audit** (2026-09-10), and **live server search** replacing prefetch-all vendor/customer pickers (2026-09-10).
- **An ambitious weighted-average-cost "Actual Profit" engine** — built in full over eight commits on 2026-09-15, then found the very next day to silently mis-cost data under this app's own deliberately-allowed "oversell" behavior, and removed in full rather than shipped in a wrong-but-plausible state. See Chapter 26 for the plain-language telling and Chapter 9.22/28 for the technical account.
- **Month-end bulk PDF ZIP download** for Invoices and Purchase Bills (2026-09-17), the latter with a choice between generated PDFs and vendors' own uploaded attachments.

One further piece of work — a self-service **"My Profile"** page for non-admin roles, closing a real gap where staff/managers had no working UI to change their own password — is implemented and verified but **IN PROGRESS — NOT YET COMMITTED AS OF 2026-09-17**; it is described throughout as

what it will be once committed, never as already shipped, per this Bible's own established convention for in-flight work.

Nothing about the system's core shape changed this window: still every page `"use client"`, still no `"use cache"` / `cacheComponents`, still the same GST math, still the same document-numbering system. This edition is, like the Fourth, a hardening and consistency pass — plus one genuinely large, honestly-reported reversal — on top of a feature set that was already largely complete.

1 Executive Summary

What is Science Hub? / What business problem does it solve? / Who uses it? / Why was it built? — Unchanged since the Fourth Edition, re-confirmed against current source. Science Hub remains a purpose-built, single-tenant business management system for one scientific-supplies trading business, covering the full commercial cycle from catalogue to GST filing, deployed once on Vercel against one Neon database. The three-role model (admin / staff / manager) is unchanged — confirmed directly against `src/lib/apiAuth.ts`'s four guards and `src/lib/sections.ts`'s six `ProtectedSection` keys, both exactly as previously documented.

What's new since the Fourth Edition, in one paragraph

Roughly two and a half weeks of continuous work (2026-09-02 → 2026-09-17) landed a GSTR-1 Offline Tool CSV export, a full second 10-specialist audit pass, a consistency fix to how products are priced end-to-end, a permanent close-out of a recurring date/timezone bug class (with a new pre-commit guard so it cannot silently regress a fourth time), the ability to correct a mis-entered payment after the fact, a real security fix moving purchase-bill attachments off public file storage onto an authenticated-only path, a month-end bulk PDF download feature, and — notably — an ambitious automatic profit-tracking engine that was built, found to be unsafe to trust on real data, and deliberately removed rather than shipped in a wrong-but-plausible state (see Chapter 26 for the full story). One further piece of work — a self-service "My Profile" page for non-admin staff — is implemented and verified but **IN PROGRESS — NOT YET COMMITTED AS OF 2026-09-17**, per `CLAUDE.md`'s own "Current Work In Progress" note; it is described below as what it will be once committed, not as already shipped.

Main features (at a glance)

Every bullet from the Fourth Edition remains accurate and is repeated below without re-labelling it "new" a second time; only bullets that are new or materially changed since 2026-08-31 carry a label.

- Invoice creation, editing, and soft-deletion, numbered in an admin-configurable layout. **NEW THIS EDITION:** New Invoice, New Purchase Bill, and the Record Return modal now show a small preview badge ("Next no.: SH-2026-27-0042") of the number a save right now would get, before the user commits to saving — a read-only preview, not a reservation (`src/lib/documentNumberPreview.ts` , 2026-09-09).
- Automatic GST computation (CGST+SGST for in-state sales, IGST for inter-state sales), server-verified independently of the browser.
- An optional per-document transportation/freight charge line, with its own GST rate and amount.
- Product returns issued as numbered credit notes, same configurable-layout numbering as invoices.
- Purchase bills from vendors with the same configurable numbering, a cancel/un-cancel workflow, and their own dedicated email-send route.

UPDATE — attachments are now considerably more secure, 2026-09-08: an uploaded vendor bill scan/photo is stored in a dedicated private Vercel Blob store rather than the original public one, and is

only ever served back to the browser through an authenticated proxy route (`GET /api/purchase-bills/[id]/attachment/[filename]`) — the raw file URL no longer works for anyone without a valid login, closing a real exposure the app had carried since attachments were first added. The bill detail/edit pages also gained an explicit download button (previously view-only).

- A single, append-only stock movement ledger recording every reason stock ever changed.
- Rate Lists — a standalone, downloadable price-sheet/catalogue builder, deliberately independent of the Product catalogue, with bulk import, PDF generation, email send, and Excel export.
- Drag-to-reorder line items on invoices, purchase bills, and rate lists.
- Bulk import of products from Excel/CSV.

NEW THIS EDITION — Correcting a payment after the fact (2026-09-07): a mistyped amount, wrong payment method, or a payment that needs deleting outright can now be edited or removed after it was recorded, on both the sales (invoice) and purchase (bill) side — closing a real, longstanding gap where the only way to "fix" a payment was to know it could never be fixed. Every correction is capped so it can never leave a credit note claiming to have refunded more cash than the business actually received.

NEW THIS EDITION — Month-end bulk PDF download (2026-09-17): the Invoices and Purchase Bills list pages can each download every document for a chosen month/year as a single ZIP of PDFs in one click. Purchase Bills additionally offers including each bill's own uploaded attachment file in the same ZIP, fetched through the same authenticated path described above rather than a raw link.

NEW THIS EDITION — GSTR-1 Offline Tool CSV export ("Tier 1, no GSP"), merged 2026-09-02: alongside the existing ZIP/ `.xlsx` GST filing package, `reports/gst-reports` can now produce a ZIP of section-wise CSVs matching the exact column format the free GST Returns Offline Tool expects — the business still files through the government's own portal/Offline Tool; this app never talks to GSTN directly.

NEW THIS EDITION — a working self-service profile page for staff and managers (IN PROGRESS — NOT YET COMMITTED AS OF 2026-09-17): clicking your own avatar now opens a dedicated "My Profile" page (name/email/password) for any role — previously that link went to the Admin Panel, which immediately bounced anyone who wasn't an admin back to the dashboard, leaving staff and managers with no working way in the UI to change their own password, even though the underlying API already supported it for any logged-in user.

- Section-based permissions so an admin can fine-tune what a given staff/manager account can view.
- Recycle Bin — 30-day grace period and automatic purge for most record types; invoices, purchase bills, and credit notes are retained indefinitely.
- Global search across invoices, customers, products, vendors, purchase bills, brands, and categories, now also matching a line item's product name inside a document, not just the document's own number or party name.

- A dedicated GST filing package, now with formula-injection hardening fully committed (it was sitting uncommitted as of the Fourth Edition).

UPDATE — the Dashboard's Cash Flow section gained real GST figures (2026-09-16): a "GST Collected" tile, a "GST Paid (ITC)" tile, and a "Net GST Payable" tile that calls the exact same GST-filing computation the GST Filing report itself uses — not a separately re-derived approximation — plus a Financial-Year + Month picker (`PeriodFilter`) that now drives the whole Dashboard financial summary. These survived a larger, since-reverted effort described below and in Chapter 26.

- Business branding, full activity log, email delivery of invoice/purchase-bill/rate-list PDFs, and a default-deny API gatekeeper (`middleware.ts`) — all unchanged.
- Client-generated idempotency keys on every document- and payment-creating route, now extended to Customer, Vendor, and Rate List creation as well (2026-09-06) — closing the last gap in this protection.

What was tried and deliberately not kept

In the interest of the same honesty this Bible has applied to every prior reversal: an ambitious automatic "Actual Profit" feature — tracking the real, weighted-average purchase cost behind every sale, so the Dashboard could show true gross profit rather than a simple sales-minus-purchases comparison — was designed, built, and shipped internally across eight commits on 2026-09-15, then found the very next day, via a live data check, to silently mis-cost any sale that this app's own deliberately-allowed "oversell" behavior touched, while still labelling that guess as verified. It was removed in full rather than patched, because the flaw was in the core method, not a display detail. The GST tiles and Financial-Year/Month picker bullet above are the genuinely useful, independently-correct pieces that survived that effort. See Chapter 26 for the plain-language version of why this was the responsible call, and `DEVELOPMENT_LOG.md`'s Era 20 for the full technical account.

Benefits

- **Accuracy** — GST, discounts, and stock arithmetic are computed once, in one place, the same way every time.
- **Accountability** — every create/edit/delete is attributed to a named user and time-stamped in the activity log.
- **Safety net** — soft-delete and the recycle bin mean mistakes are recoverable, not catastrophic.

UPDATE — Correctability, not just recoverability: a mistake made while recording a payment no longer needs a workaround or a support request to fix — it can be corrected or removed directly, with the same safeguards (transactional integrity, a floor against over-crediting) as creating it in the first place.

UPDATE — Confidentiality of uploaded documents: a vendor's bill scan or photo, once uploaded, can no longer be viewed by anyone who merely obtains its old public link — only a logged-in user going

through the app itself can retrieve it now.

- **Compliance-readiness** — the GST filing package is built directly from the same transactional data used to run the business, now including a CSV format the free government Offline Tool can import directly.
- **Access control that matches how a small business is actually staffed** — and, once the in-progress profile page above is committed, every role, not only admins, will have a working screen to manage their own login credentials.
- **Time saved at month-end** — a whole month's worth of invoices or purchase bills, including their attachments, can be pulled down as one ZIP file instead of one-by-one downloads.
- **Resilience against duplicate submissions and stale UI** — idempotency keys now covering every create route in the app including Customer/Vendor/Rate List.
- **Engineering discipline as a genuine benefit, not just a slogan** — this edition's own costing-engine reversal is direct evidence that a feature is shipped only once it can be trusted, not merely once it looks plausible on screen.

2 High-Level System Overview

The eight-step request/response flow and the three-layer architecture table from the Fourth Edition are unchanged and re-confirmed against current source: every page under `src/app/(dashboard)` is still explicitly marked `"use client"` (spot-checked across the newest files added this window, including `src/app/(dashboard)/profile/page.tsx`), there is still no async server component fetching data, and `"use cache"` / `cacheComponents: true` remain confirmed-forbidden in this codebase.

The three big layers — unchanged

LAYER	TECHNOLOGY	RESPONSIBILITY
Presentation (Frontend)	Next.js 16 (App Router) + React	Everything the user sees and clicks — forms, tables, dashboards, PDFs on screen.
Application (Backend)	Next.js API Routes (same project)	Business rules: validation, GST math, document numbering, the stock ledger, permission checks, email sending, idempotency checks.
Data (Database)	PostgreSQL on Neon, accessed via Prisma ORM	The permanent record of every customer, product, invoice, bill, rate list, and user.

One genuinely new moving part: an authenticated file-access path

Every prior edition's diagram of "browser → API route → Prisma → Neon" was already accurate for every kind of data except one: an uploaded purchase-bill attachment used to be handed to the browser as a plain public file-storage URL, which — once known — worked for anyone, logged in or not. As of 2026-09-08 that path changed shape, verified directly against `src/lib/blobStorage.ts`, `src/lib/attachmentHref.ts`, and the two route handlers under `src/app/api/purchase-bills/`:

1. New attachment uploads go to a second, separate Vercel Blob store created with `access: "private"` — a private blob simply does not resolve for a request that isn't presenting the store's own access token, regardless of whether the requester is logged into Science Hub or not.
2. The application itself is now the only thing able to fetch a private blob's contents, via `@vercel/blob`'s server-side `get()` call, using a token resolved through a small fallback chain in `getPrivateBlobToken()` (`LIVE_PRIVATE_BLOB_READ_WRITE_TOKEN` → `DEV_PRIVATE_BLOB_READ_WRITE_TOKEN` → `PRIVATE_BLOB_READ_WRITE_TOKEN`) — needed because Vercel forces separately-prefixed environment-variable names once a second Blob store is connected to one project.
3. The browser is only ever given one of two app-owned URLs that end in the file's own name: `GET /api/purchase-bills/[id]/attachment/[filename]` for an already-saved bill's current attachment, or `GET /api/purchase-bills/attachment/[folder]/[filename]` for a file that was just uploaded on the Edit form but not yet attached to a saved bill.
4. The business's own logo is deliberately left on the original public store, unchanged — it has to be publicly readable, since it appears on the unauthenticated login page.

This is a real architectural addition, not a cosmetic one: it is the first place in the application where "does the browser have this URL" and "is this content actually accessible" have ever been different questions, and it is the direct, verified fix for a real confidentiality gap the app had carried since attachments were introduced (see Chapter 19 for the security write-up and [DEVELOPMENT_LOG.md](#) Era 19.5 for the full four-commit history, including a same-day production outage the environment-variable rename itself caused, and its fix).

Nothing else about the system's shape changed this window: there is still no separate backend server to deploy, Vercel still builds and hosts the Next.js project as one unit, and every other kind of data still flows exactly as the eight-step diagram in prior editions describes.

3 Complete Technology Stack

Every technology below is confirmed present in `package.json` or the source code directly. Unchanged in substance from the Fourth Edition (2026-08-31) except where called out below.

Core Framework

TECHNOLOGY	WHAT IT IS	WHY IT'S USED HERE
Next.js 16	A framework built on top of React that handles routing, bundling, and server/API code, all in one project.	Lets one project serve both the web pages and the API that powers them, deployed as a single unit on Vercel.
React 19	A JavaScript library for building interactive user interfaces out of reusable "components".	Every screen in Science Hub is built from React components.
TypeScript	A version of JavaScript that adds type-checking.	Used throughout the codebase for safer, self-documenting code.

Data & Storage

TECHNOLOGY	WHAT IT IS	WHERE/WHY IT'S USED
PostgreSQL	A mature, industry-standard relational database.	The single source of truth for every business record.
Neon	A cloud hosting service that runs PostgreSQL for you.	Hosts the production database; a retrying migrate-deploy wrapper (unchanged since 2026-08-21) absorbs Neon's cold-start latency.
Prisma	An "ORM" for writing database queries as regular code.	<code>prisma/schema.prisma</code> now defines 22 data models (up from 21 at the Fourth Edition).
Vercel Blob	A cloud file-storage service.	UPDATE THIS EDITION — now backed by two separate Blob stores, not one. See below.

UPDATE — two-store Blob model, 2026-09-08 (commits [0104478](#) / [c0f0698](#) / [9b97850](#) / [652bf50](#)): As of the Fourth Edition, "Vercel Blob" meant one public-access store holding both the business logo and every purchase-bill attachment — meaning a purchase-bill attachment's raw URL was readable by anyone who obtained it, no login required. This edition, purchase-bill attachments were moved to a second, dedicated store created with `access: "private"`, while the original store stays public and keeps holding only the business logo (which must stay publicly readable, since it's shown on the unauthenticated login page). A purchase-bill attachment is now only ever reachable through one of two authenticated proxy routes, never a raw blob link. `src/lib/attachmentHref.ts` is the one place either URL shape gets built. Full env-var detail is in Chapter 23; the same-day production outage this migration caused is in Chapter 21.

Prisma schema, **22 models** (verified directly against `prisma/schema.prisma`): the net change since the Fourth Edition's 21 is a single new model, `NotificationDismissal` (backing the Notification Bell's per-user "dismiss" state — see Chapter 8). Separately, a weighted-average-cost "Actual Profit" costing engine was built and then fully removed after a live data-corruption bug over 2026-09-15/16 (see `DEVELOPMENT_LOG.md` Era 20) — a direct schema diff across that whole arc returns zero lines, confirming the revert left no residual field, column, or model behind. The three `GENERATED ALWAYS AS (...) STORED` columns are unchanged. `PurchaseBill` also gained a plain `attachmentSize Int?` column this window (2026-09-07) — the uploaded attachment's byte size, captured at upload time.

Authentication & Security

Unchanged: NextAuth v4, bcryptjs (cost factor 12), AES-256-GCM via `src/lib/crypto.ts`.

Communication & Documents

TECHNOLOGY	WHAT IT IS	WHERE/WHY IT'S USED
Nodemailer	Sends email.	Unchanged at 9.0.5 since the Fourth Edition.
jsPDF	Builds a PDF file entirely inside the browser.	UPDATE — the per-page canvas export switched from JPEG to PNG and jsPDF's <code>"SLOW"</code> compression flag was added (2026-09-09), since a flat-background, sharp-edged canvas render compresses far better losslessly than as a JPEG.
html2canvas	Captures the on-screen document layout as an image.	Unchanged.
ExcelJS	Generates/parses <code>.xlsx</code> files.	Unchanged.
JSZip	Builds <code>.zip</code> archives.	Bundles GST filing — and, new this window, the month-wise "Bulk Download (ZIP)" button on Invoices/Purchase Bills (<code>src/lib/bulkPdfDownload.ts</code> , 2026-09-17). Confirmed unchanged at <code>^3.10.1</code> — a new internal caller, not a new package.

Styling & UX, Hosting & Deployment

Unchanged (CSS Modules + CSS Custom Properties; Vercel; `npm`'s mandatory `postinstall`). Vercel now provisions three separate Blob-related env vars instead of one — see Chapter 23.

Not present in this codebase, still: no Redis, no Railway hosting, no AI/Gemini bill-extraction, no separate object-storage provider outside Vercel Blob — the new private store is a second store within the same Vercel Blob service, not a new third-party dependency.

4 Folder Structure

All application code lives under `src/`. The overall shape is unchanged from the Fourth Edition.

```
src/
  app/
    (dashboard)/
      profile/          NEW THIS EDITION, NOT YET COMMITTED – see below
      ...               (dashboard/, sales/, purchases/, products/, brands/, categories/,
                        reports/, admin/, bin/, settings/ all unchanged)
    api/
      purchase-bills/[id]/attachment/[filename]/      NEW THIS EDITION
      purchase-bills/attachment/[folder]/[filename]/  NEW THIS EDITION
      ...       (private-Blob-serving proxy routes, see Ch. 3/22/23)
    login/, forgot-password/, reset-password/, find-email/  Public, unauthenticated pages
  components/
    ui/              ...and (new this edition) BulkDownloadOptionsDialog
    dashboard/       NEW THIS EDITION – PeriodFilter.tsx
    dialogs/         ...and (new this edition) BulkDownloadOptionsDialog
    profile/         NEW THIS EDITION, NOT YET COMMITTED – MyProfileCard.tsx
  lib/
    documentNumberPreview.ts  new this edition (Ch. 9/13)
    blobStorage.ts            extended for the two-store model (Ch. 3)
    attachmentHref.ts         new this edition (Ch. 3)
    bulkPdfDownload.ts        new this edition
    useEntitySearch.ts        new this edition
    validationCsv.ts          new this edition (Ch. 15)
  prisma/
    schema.prisma            22 models (up from 21), plus one-off migration scripts used once
                                during the private-Blob migration and left in the tree as a
record
                                of that migration, not for repeated use.
  scripts/
    check-date-safety.js      NEW THIS EDITION (2026-09-09) – greps staged/all .ts/.tsx files
for
                                the two shapes of the recurring IST/UTC date-boundary bug and
blocks
                                the commit if either is found. Runnable via "npm run
check:dates".
```

What's new here since the Fourth Edition

- `src/app/api/purchase-bills/[id]/attachment/[filename]/route.ts` and `.../attachment/[folder]/[filename]/route.ts` — the two authenticated proxy routes serving every purchase-bill attachment from the new private Blob store (2026-09-08).
- `src/lib/attachmentHref.ts` (2026-09-08) — the one place the two attachment-serving URL shapes are constructed.
- `src/components/dashboard/PeriodFilter.tsx` — a shared dashboard FY+month selector (part of the now-reverted costing engine's work, but confirmed still present in the tree — it was not removed along with the rest of that engine).

- `src/lib/useEntitySearch.ts` (2026-09-10) — a debounced, abort-cancelled hook replacing a prefetch-the-whole-table-then-filter-client-side approach for the Purchase Bill vendor picker and Invoice customer picker.
- `src/lib/bulkPdfDownload.ts` (2026-09-17) — the shared month-wise "Bulk Download (ZIP)" logic, built on the JSZip dependency already present as of the Fourth Edition.
- `src/components/dialogs/BulkDownloadOptionsDialog.tsx` (2026-09-17) — the Purchase Bills-only popup for choosing Generated PDFs and/or Attached Files before a bulk download.

IN PROGRESS — NOT YET COMMITTED AS OF 2026-09-17:

`src/app/(dashboard)/profile/page.tsx` and `src/components/profile/MyProfileCard.tsx` (`git status` shows both directories as untracked) — a standalone "My Profile" page giving staff/manager roles a working UI to change their own name/email/password. Also uncommitted:

`prisma/migrations/20260917000000_add_activity_log_entity_id_action_index/`, adding a supporting index for `ActivityLog` queried by `entityId` / `action` (the Bin page and Credit Notes list both query this table by entity id with no index today).

5 Application Flow

The thirteen-step typical-session flow (visit → login → session cookie → dashboard → sidebar navigation → create/edit → server validates and writes inside a transaction → PDF/print/email → activity log → soft-delete-to-bin) is unchanged in shape since the Fourth Edition, and the flow-level conventions confirmed there (every list screen paginated server-side with a matching `/stats` endpoint; every popup dialog the same `Modal` component in `variant="fullscreen"`) remain accurate.

What changed this edition

- **A next-document-number preview joins the create flow.** Before a user even starts filling in New Invoice, New Purchase Bill, or the Record Return modal, a small badge now shows the number a save right now would receive (`src/lib/documentNumberPreview.ts`, 2026-09-09) — a read-only preview, not a reservation. See Chapter 9.1/10/13 for the full mechanism.
- **A document's position in a list is now governed by when it was created, not by its own editable date.** Reordering an already-created document (via a later backdate/postdate) no longer moves it on the Invoices/Purchase Bills/Credit Notes lists — see Chapter 9.1 for the full reasoning.
- **Recording a payment is no longer a one-way action.** Steps that used to end at "payment recorded, done" now have a corresponding correction path: a mistyped amount, wrong method, or an unwanted payment can be edited or deleted from the same Payment History table it was recorded in (Chapter 9.4/9.8).
- **A purchase-bill attachment now flows through an authenticated proxy, never a public link,** from the moment it's uploaded to the moment it's viewed or included in a bulk download (Chapter 2/19).
- **Live server search replaces a prefetch-then-filter step** on the vendor picker (Purchase Bill) and customer picker (Invoice) — typing now queries the server directly rather than filtering an already-downloaded table client-side (Chapter 14).
- **A month-end bulk step was added:** instead of opening and downloading each invoice/purchase-bill PDF one at a time, an accountant can pick a month/year on the list page and download every matching document as one ZIP (Chapter 9.1/9.7).

Every other mechanic described in prior editions — the idempotency-key-guarded submit, the Serializable-transaction-with-retry number generation, the server-independent `isInterState` derivation, and the Recycle Bin's soft-delete behavior — is unchanged and remains accurate as previously documented.

6 User Roles & Permissions

Science Hub has three roles, stored as a plain text field (`User.role`): `admin`, `staff` (the default for new users), and `manager`. This three-role model, the six-key section-permission system, and the enforcement summary are unchanged in shape since the Fourth Edition and remain verified accurate against both `src/lib/apiAuth.ts` and `src/lib/sections.ts`.

6.1–6.5 — Unchanged

Admin (unrestricted, auto-passes every section check), Staff (default, full read/write, restricted only by the six section-permission keys), Manager (functionally identical to staff for visibility, hard-blocked from every mutation via `requireWriteAccess()` / `useCanWrite()`), the six `ProtectedSection` keys, and the four-guard enforcement summary (`requireSession()`, `requireAdmin()`, `requireWriteAccess()`, `requireSectionAccess(section)`) are all unchanged since the Fourth Edition.

6.6 NEW THIS EDITION — Dedicated manager-role permission audit (2026-09-17)

`CLAUDE.md`'s Features Completed #60 records a dedicated audit that seeded a throwaway manager-role user via the real API, drove it through Playwright against the running dev server, then hard-deleted the seeded user afterward. Every finding below was independently spot-checked against the current source for this edition:

- The sidebar still has no nav entry for Settings/Admin/Permissions unless `session?.user?.role === "admin"` — confirmed by reading `NAV_GROUPS` directly.
- `admin/page.tsx` still bounces any non-admin session straight to `/dashboard` — confirmed at line 140.
- Section-gated pages/routes correctly admit or reject a manager based on their live `SectionPermission` grants, and server-side mutation attempts as manager (delete an invoice, edit a customer, adjust stock, list `/api/admin/users` / `/api/admin/permissions`) are all rejected with 403 — confirmed sound, no code change was needed, and none was made. The permission system held up.

6.7 NEW THIS EDITION, IN PROGRESS — NOT YET COMMITTED AS OF 2026-09-17 — a real gap the audit did find: no profile page for non-admin roles

The one genuine gap the audit surfaced: clicking your own avatar (both the sidebar user block and the topbar chip) unconditionally linked to `/admin` for every role. Since `/admin` immediately redirects any non-admin away (6.6 above), staff and manager had no working UI at all to change their own name, email, or password — despite the underlying `GET / PUT /api/admin/profile` route already being fully role-agnostic (it calls `resolveSessionUser()`, gated only on `getServerSession()`, with no role check anywhere in the handler).

The fix, verified directly against the working tree (`git status` shows all of the following as modified/untracked, not yet committed):

- `src/components/profile/MyProfileCard.tsx` is a new shared component — the "My Profile" view-plus-edit card that used to live inline on `admin/page.tsx`. `git diff --stat HEAD` confirms the extraction cleanly: 182 lines removed, 3 lines added on `admin/page.tsx`. Admin's own profile section is visually unchanged — it is the identical card, just componentized.
- `src/app/(dashboard)/profile/page.tsx` is a new standalone page, reachable by every authenticated role, with no extra route guard needed.
- `DashboardShell.tsx`'s two avatar links now route conditionally: `session?.user?.role === "admin" ? "/admin" : "/profile"`. The mobile topbar's page-label logic special-cases `/profile` → "My Profile", since it's not itself a `NAV_GROUPS` entry.

A second, related fix in the same uncommitted diff: `CLAUDE.md`'s own Feature #60 writeup flags a cosmetic rough edge it explicitly did not fix — saving your own profile triggers a `session.update()` call that transiently flips NextAuth's `useSession()` status to `"loading"`, which used to flash the entire dashboard shell to a bare full-page "Loading..." screen for about a second. The current working tree already carries a fix for this, not reflected in that `CLAUDE.md` note: a one-way `hasAuthenticatedOnce` latch now gates the full-page loader, so a later transient "loading" status from an in-session `update()` call no longer blanks the shell.

Also uncommitted, in the same working tree: `src/app/api/admin/profile/route.ts` and the self-edit path of `src/app/api/admin/users/[id]/route.ts` both gained a `rateLimit("profile-password:<id>", 10, 15 * 60 * 1000)` check before a password change proceeds — see Chapter 19.1.

7 Authentication & Sessions

7.1 How login works

Unchanged. `git log -- src/lib/auth.ts` shows no commit since `66177dc` (2026-08-21) — well before the Fourth Edition's own 2026-08-31 cut, and confirmed via `git diff HEAD` to carry no uncommitted changes either. Every mechanism (lowercase/trim, NextAuth Credentials `authorize()`, dual rate limits, bcrypt comparison, a signed JWT, an `httpOnly` /secure 8-hour cookie) is current, unmodified behavior.

7.2–7.3 What's inside the session, Cookies and JWT

Unchanged: `id`, `name`, `email`, `role`, `sections`. Section grants reload every 30 seconds; a full account-validity check happens at most every 5 minutes. A signed, tamper-proof `httpOnly`, Secure-in-production cookie.

7.4 Password reset flow

Unchanged five-step flow. The same invalidate-on-change mechanism is now also user-triggered from the new `/profile` page's own Change Password form (Chapter 6.7) — `MyProfileCard.tsx`'s `savePassword()` explicitly signs the user out right after a successful change, since the server bumps `tokenVersion` and would otherwise desync this browser's own current session silently.

7.5–7.7 Password storage, Authorization vs. authentication, The middleware gatekeeper

All unchanged. `PUBLIC_API_PREFIXES` / `PUBLIC_API_EXACT` in `middleware.ts` has not silently grown — no diff to that file, committed or uncommitted, since the last edition's check.

8 Database (Full ER Reference)

The database has **22 tables** ("models" in Prisma) defined in `prisma/schema.prisma` — **up from 21 in the Fourth Edition**. The single new model is `NotificationDismissal`, which backs the topbar Notification Bell's per-user dismissal state. There are still no database-level enum types in this schema.

A costing engine was built and fully reverted — net effect on the schema is zero, but worth recording. Between the Fourth Edition and now, a weighted-average-cost "Actual Profit" engine was built (`InvoiceItem.costPrice / costSource`, `ReturnItem.costPrice`, four migrations dated 2026-09-15) and then **fully removed the next day** (`20260916000000_remove_wac_costing_columns`) after a live QA pass found it silently mis-costing oversold units while still labeling the result "verified." Reading the current `InvoiceItem` and `ReturnItem` model blocks confirms these columns are gone. These columns were added then dropped again — net effect on the schema is zero, but worth recording since a future session might otherwise re-propose the same columns without knowing they were tried and reverted; see `DEVELOPMENT_LOG.md` Era 20. **Do not re-attempt a costing/profit feature without reading that entry in full.**

8.1 Soft-delete vs. permanent records

Unchanged since the Fourth Edition, re-confirmed:

HAS A "RECYCLE BIN" (<code>DELETEDAT</code>)	NEVER SOFT-DELETED (PERMANENT/APPEND-ONLY)
Customer, Category, Brand, Product, Vendor, PurchaseBill, RateList, Invoice, Return	User, PasswordResetToken, ActivityLog, InvoiceItem, Payment, ReturnItem, BusinessSettings, PurchaseBillItem, PurchasePayment, StockMovement, SectionPermission, RateListItem, NotificationDismissal

`NotificationDismissal` joins the permanent/append-only side, but not in the same sense as `ActivityLog` — it has no `deletedAt` column at all; it is opportunistically purged past a 7-day rolling window on read (`purgeExpiredDismissals()`).

8.2 New model: `NotificationDismissal`

```
model NotificationDismissal {
  id          String  @id @default(cuid())
  userId      String
  user        User    @relation(fields: [userId], references: [id], onDelete: Cascade)
  category    String  // "stock" | "overdueInvoices" | "overdueBills" | "overLimitCustomers"
  | "binExpiring"
  entityId    String  // the underlying record's id
  dismissedAt DateTime @default(now())

  @@unique([userId, category, entityId])
  @@index([userId, dismissedAt])
}
```

Records that a specific user cleared a specific notification card — the notification itself is never persisted; only the fact of the dismissal is. Reads check `dismissedAt` against a rolling window (24h) rather than mere row existence, so an alert that is still true tomorrow resurfaces instead of staying hidden forever once dismissed once.

8.3 Other field changes since the Fourth Edition

Cross-checked directly against `prisma/schema.prisma`:

- `Customer.creditLimit` — nullable `Float`. Null means no limit enforced; checked against outstanding balance on invoice create/edit, inside the same transaction as the write it guards — not a pre-check outside the transaction, closing a TOCTOU window two concurrent invoices could otherwise jointly exploit.
- `Customer.idempotencyKey` / `Vendor.idempotencyKey` / `RateList.idempotencyKey` — all nullable `String? @unique`, same client-generated-key pattern already used by Invoice/PurchaseBill/Payment/PurchasePayment/Return.
- `PurchaseBill.attachmentSize` — nullable `Int`. Byte size of the uploaded vendor-bill attachment, captured at upload time.

Also confirmed present: `Product.listPrice` (`Float`, NOT NULL) and `Product.discountPercent` (`Float @default(0)`). Note that `Product.purchasePrice` itself remains a **nullable** `Float?` at the database level — "mandatory" is an application-layer rule, not a DB `NOT NULL` constraint.

8.4 New indexes since the Fourth Edition

`NotificationDismissal` 's own two indexes (new only because the model is new); the three new `idempotencyKey` columns each carry an implicit unique index.

IN PROGRESS — NOT YET COMMITTED: `ActivityLog` gains `@@index([entityId, action])` in the working tree, paired with an untracked migration folder. `ActivityLog` 's two pre-existing indexes (`userId`, `createdAt`) are unaffected. This index does not exist in any deployed/committed state as of this edition.

8.5 Key relationships, plain English

Unchanged since the Fourth Edition. `NotificationDismissal` adds one new edge: one User to many NotificationDismissals (cascade-deleted with the user), with no relation back to the entity being dismissed beyond the free-text `entityId` string.

8.6 CLAUDE.md cross-check — one real inconsistency found

`CLAUDE.md` 's own "Database Models (current)" section was cross-checked field-by-field:

- `Customer.creditLimit` and `Customer.idempotencyKey` — **correctly documented**.

- `PurchaseBill.attachmentSize`, `Vendor.idempotencyKey`, `RateList.idempotencyKey` — **not listed** in their respective model bullets, even though sibling fields added the same day are documented. The `Vendor` bullet is also missing `city` / `state` / `pincode` fields the current schema actually carries.
- `NotificationDismissal` — **not listed at all**, despite being a real, committed model referenced by name elsewhere in `CLAUDE.md`'s own Features Completed narrative.

None of these are schema bugs — the database itself matches every claim verified in this chapter. They are documentation-completeness gaps in `CLAUDE.md`'s model list, flagged here rather than silently corrected.

9 Complete Module Documentation

9.1 Invoices

Purpose, numbering, place-of-supply-mandatory, no-duplicate-product-lines, server-independent `isInterState`, transportation charge, FY-bounded editable date, and the editable "Bill To" customer are all unchanged in shape since the Fourth Edition.

NEW THIS EDITION — Next-invoice-number preview badge (2026-09-09) — the New Invoice page now shows a small blue badge ("Next no.: SH-2026-27-0042") previewing the number a save right now would receive. Backed by `GET /api/invoices/next-number ( requireSession() )`, delegating to a shared `previewNextDocumentNumber()` helper — verified directly: it runs the *exact same* `numberFormatDbFilter()` / `computeNextNumber()` computation the real create route runs, just without the transaction, the insert, or consuming the one-time override. Deliberately a preview, not a reservation. Extended to New Purchase Bill (`?billDate=`) and the Record Return modal (`?date=`), both refetching whenever their date field changes.

NEW THIS EDITION — Return-date display bug fixed, and "newest first" sort switched (back) to createdAt (2026-09-09) — a production report showed an invoice's own date disagreeing with its "Created by" sub-line; investigation found this was a *deliberate* backdate, not a timezone bug, but surfaced two real fixes. First: the invoice detail page's Credit Note history table and its embedded PDF-print equivalent were both showing the credit note's system creation timestamp under a column meant to show its own `date` — both now show `formatDate(ret.date)`. Second, and more consequential: every "newest"/"oldest" sort on Invoices, Purchase Bills, and their sub-tables was switched to sort by `createdAt` rather than the document's own editable `date` / `billDate`. This reverses an earlier same-session decision after the reasoning was reconsidered: document numbers are always assigned in creation order, so `createdAt` order is *always* identical to numbering order, while a document's own date is editable and routinely backdated. Sorting by the editable date instead lets a backdated document visibly reorder away from where its own number sequence places it — confirmed as the literal cause of a real historical-backfill batch (51 purchase bills entered over 3 days, backdated up to 5 months) looking "jumbled" on the list.

NEW THIS EDITION — "Created/edited" time-of-day sub-lines removed outright (same commit) — deleted, not relabeled, from the Invoices list, Purchase Bills list, Credit Notes list (and its Excel export's "Time" column), and the Customer detail page's invoice sub-table. GST filing, financial year, and document numbering all operate purely on the calendar day, so a bare creation/edit time next to a document's own (possibly backdated) date has no legal meaning and reads as if it belongs to the wrong date. Time-of-day is intentionally kept in exactly three places: the Admin Activity Log, the Stock Movement Ledger export, and the Notification Bell.

NEW THIS EDITION — Month-wise bulk PDF ZIP download (2026-09-17) — the Invoices list toolbar gained a "Bulk Download (ZIP)" button next to the Month/Year filter, enabled once both are selected. On click it fetches every matching row at `pageSize=2000 ( sort=oldest )`, renders one PDF per

row sequentially through the existing hidden-iframe render + IndexedDB PDF cache pipeline, and packs the results into a ZIP (`src/lib/bulkPdfDownload.ts` , dynamically imports `jszip`). A document number containing "/" is sanitized to "-" before use as a zip entry name. A per-row failure doesn't abort the batch; the zip still downloads with whatever succeeded, and the toast reports a partial-failure count.

9.2 Customers

Unchanged in shape. Idempotency-key protection on create remains as documented.

9.3 Products, Brands, Categories

Unchanged since the Fourth Edition's own coverage of the List Price/Purchase Price consistency work.

9.4 Payments (Received)

Unchanged core behavior — `amount` /status always recalculated from the true sum of all payment rows, never incrementally adjusted.

NEW THIS EDITION — Edit and Delete for an existing payment (2026-09-07) — closes a real, longstanding gap: `PUT /api/invoices/[id]/payment/[paymentId]` already existed but had **no UI wired to it anywhere**. This edition adds `DELETE /api/invoices/[id]/payment/[paymentId]` ; an Actions column (`canWrite` -gated edit-pencil/delete-trash icons) on the invoice detail page's Payment History table (Edit opens a fullscreen `Modal` at `maxWidth="26rem"` , Delete goes through the standard `ConfirmDialog`); and **a real data-integrity guard, not previously enforced anywhere**: a credit note is capped against an invoice's `paidAmount` only at its own *creation* time — a point-in-time check, not a standing constraint. Reducing or deleting a payment afterward could otherwise leave an invoice claiming to have refunded more cash via credit note(s) than it ever actually received. Both `PUT` and `DELETE` now re-verify `newPaidAmount >= existingReturnTotal` inside the same transaction.

Two real gaps closed in a follow-up pass the same week, triggered by the user's own review: (1) the Edit Payment Save button had no dirty-check, silently re-saving identical values and logging a spurious `ActivityLog` entry every time; fixed with the same `useDirty()` hook Rate List Edit already used. (2) The invoice detail page's three payment handlers had no `try/catch` around their `fetch()` calls — a genuine network failure left the saving/deleting flag stuck `true` forever, with the `OverlayLoader` never clearing. Pre-existing gap in `handleAddPayment` specifically, carried forward when the Edit/Delete handlers copied its shape; the purchase-bill side already had the correct pattern. Both fixes were live-verified via Playwright, including an actually-aborted in-flight request to prove the network-error path.

NEW THIS EDITION — Shared payment-method list (2026-09-06) — `PAYMENT_METHODS = ["Cash", "UPI", "IMPS", "NEFT", "RTGS", "Cheque", "Card", "Other"]` , one module now backing the Invoice, Purchase Bill, and standalone `RecordPaymentDialog` forms — previously three separately hand-rolled lists that could drift. `IMPS` is new; picking "Other" reveals a required free-text field.

9.5 Returns / Credit Notes

Unchanged business rules. The credit-note-creation route gained a TOCTOU fix: `POST /api/invoices/[id]/returns` 's paid-amount and remaining-quantity caps were previously validated against a snapshot fetched *before* the route's own Serializable transaction opened — a concurrent payment edit/delete landing in that gap went undetected. `attemptCreate()` now re-reads `invoice.paidAmount` and item quantities inside the transaction itself.

9.6 Vendors

NEW THIS EDITION — Inactive-vendor indicator (2026-09-10) — the purchase-bill vendor picker now checks `Vendor.isActive` and shows an "Inactive" tag in the dropdown plus a warning under the selected vendor's summary when false, so staff notice before billing against a deactivated vendor.

NEW THIS EDITION — Live server search replaces the prefetch-all vendor/customer picker (2026-09-10) — Purchase Bill's vendor picker and Invoice's customer picker previously fetched the entire table at `?pageSize=5000` (clamped to 2000 server-side) and filtered client-side per keystroke, silently dropping anything past the cap once the real row count grew past it. `useEntitySearch()` is a debounced (250ms), `AbortController`-cancelled hook that queries the existing paginated list routes live — mirroring `GlobalSearch` 's own pattern, down to sending `headers: { "x-no-loader": "1" }` so a per-keystroke lookup doesn't trigger the app's global loading indicator.

9.7 Purchase Bills

Numbering, transport charge, cancel/un-cancel workflow, and FY-bounded `billDate` editing are unchanged in shape.

NEW THIS EDITION — Attachments moved to a private Vercel Blob store (2026-09-08) — the original attachment store had been public since inception, meaning the raw blob URL was readable by anyone who obtained it, no login required. New uploads now go to a dedicated **private**-access store; the business logo deliberately stays on the original public store. Attachments are only ever served through an authenticated proxy — `GET /api/purchase-bills/[id]/attachment/[filename]` for a saved bill, `GET /api/purchase-bills/attachment/[folder]/[filename]` for a just-uploaded, not-yet-saved replacement — because a browser names a file saved from an inline viewer after the URL's last path segment, not the `Content-Disposition` header. The bill detail/edit pages gained an explicit download button (previously view-only). One real production outage followed the same day: Vercel renamed the production-scoped Blob token as part of connecting the second store, but the app's own fallback chain still checked the old name first, so production silently fell through to the *dev* store's token and every attachment fetch 403'd — fixed by reordering the token-lookup chain.

NEW THIS EDITION — `PurchaseBill.attachmentSize` display (2026-09-07) — a nullable `Int` column capturing the byte size at upload time, shown on the bill detail page (e.g. "invoice.pdf · 240 KB").

NEW THIS EDITION — CGST/SGST-per-line footing fix, mirroring the invoice-side fix —

`splitGstForDisplay(totalGst)` rounds the combined GST total once, then gives SGST whatever remains after CGST's own independent rounding, so the two printed halves always foot exactly to the printed Grand Total. Originally fixed on the invoice side; a grep confirms `splitGstForDisplay` is now called from both the invoice and purchase-bill detail pages — the purchase-bill side had not been routed through it until this pass.

NEW THIS EDITION — Bulk-download options dialog (2026-09-17) — unlike Invoices' one-click

ZIP download, Purchase Bills' "Bulk Download (ZIP)" button opens a small popup (mirroring `PdfCopyDialog`'s own overlay structure, not the shared fullscreen `Modal`) offering two independent checkboxes — **Generated Bill PDFs** (checked by default) and **Attached Bill Files** (unchecked by default, fetched via the same authenticated attachment route, never a raw blob URL). Both can be selected together; the dialog disables Download with "Select at least one option" when neither is checked. A bill with no attachment is silently skipped from that half of the zip (counted, not treated as a failure).

9.8 Payments (Made)

Mirrors the sales-side payment logic exactly, and gained the identical Edit/Delete feature in the same window (2026-09-07) — verified against `src/app/api/purchase-bills/[id]/payment/[paymentId]/route.ts` (this route did not exist at all in the Fourth Edition's window, only `POST` did). Both `PUT` and `DELETE` open a Serializable transaction with a P2034 retry loop, re-fetch the bill and remaining payment rows inside the transaction, and recompute `paidAmount` from that full set. A **cancelled bill's `status` is preserved verbatim** across the recompute rather than being silently flipped back to unpaid/partial/paid. There is no equivalent credit-note-floor guard on this side, since there is no `PurchaseReturn` model — confirmed as an intentional, documented asymmetry rather than an oversight.

9.9 Stock & the Stock Movement Ledger

Confirmed unchanged, verified directly by reading `src/lib/stockMovement.ts` in full. The closed, specific `StockMovementType` union is exactly the same set documented in the Fourth Edition. `batchAdjustStock()` itself is unchanged: it still ends with a single `createMany()` call and returns immediately — there is no call into any costing/recost module anywhere in the function.

This is worth stating plainly rather than silently, because of what happened in between: an 8-commit, single-day effort (2026-09-15) built a full weighted-average-cost engine — `src/lib/inventoryCosting.ts`'s `recostProducts(tx, productIds)` — wired to run *from inside* `batchAdjustStock()` on every stock-affecting mutation. A live QA pass the next day (2026-09-16) reproduced a sale that physically came entirely from a ₹50/unit purchase batch being costed at ₹100/unit — and the result was still labeled "ledger" (verified). Root cause: this app deliberately allows overselling past available stock, and the chronological-replay algorithm had no correct way to attribute a cost to units sold before enough matching purchase quantity existed to cover them. Because the bug lived in the core algorithm rather than a display edge case, the entire feature was removed in commit

`eee89b8`, cleanly, with zero residue. Anyone revisiting a costing/profit engine for this app should read that commit's own description before starting.

Three real, unrelated bug fixes survived the same removal commit: the `Customer.creditLimit` "Infinity" -string rejection gap, the purchase-bill `splitGstForDisplay()` fix described under 9.7, and a `.webp` attachment-upload magic-byte check that previously only verified the RIFF container and not the `WEBP` fourcc.

9.10 Reports

NEW THIS EDITION — Dashboard Cash Flow gained real GST tiles — a genuine, still-shipped feature that survived the Era 20 costing-engine removal (it doesn't depend on per-line cost tracking at all): "GST Collected" and "GST Paid (ITC)" tiles, plus a "Net GST Payable" tile that calls `buildGstFilingReport()` directly — the same figure and computation as the GST Filing report's own line, not a separately re-derived approximation. Gated by `requireGstFilingAccess()` (centralized into `src/lib/apiAuth.ts` in the same window).

NEW THIS EDITION — PeriodFilter, an FY+month selector for the Dashboard's financial figures — backs `GET /api/reports?type=combined-dashboard&fy=...` and `?type=financial-years / ?type=net-gst-payable`; month drill-down is done client-side from the returned `financials.monthly` array rather than triggering a refetch per month selected.

9.11 GST Filing

UPDATE — formula-injection hardening now committed (2026-09-05/09) — the hardening flagged "in progress, uncommitted" in the Fourth Edition has landed and been consolidated further. `neutralizeFormulaCell()` is now invoked from a single shared builder, `src/lib/validationCsv.ts`'s `buildValidationCsv()`, used by both `gstFilingZip.ts` and `gstr1CsvZip.ts` for their "Validation-Report.csv" output — previously each ZIP builder carried its own copy of this logic.

FIXED — transport-charge GST under-reporting (`src/lib/gstFiling.ts`, 2026-09-09) — an invoice's Transport Charge GST was previously excluded entirely from "Net GST Payable" and from `salesRegisterByRate` — a mixed-rate invoice carrying a transport charge under-reported by exactly the transport portion. The fix accumulates dedicated `transportOutput*` running totals and folds them into the same-invoice rate bucket and into the headline output-tax total.

FIXED — HSN Summary / headline-total exclusion consistency, same commit — an invoice whose place-of-supply doesn't resolve to a recognized GST state (or a credit note with no assigned number) was correctly dropped from the GSTR-1 CSV export, but not from the HSN Summary sheets or the on-screen/JSON/xlsx "Net GST Payable" summary — so the headline number could overstate what the business's own CSV export actually files, with no on-screen warning. All three outputs now agree on what's excluded.

Everything else in this section — Sales/Purchase Register, the B2B/B2C split, Credit Notes, HSN Summary, ZIP/ `.xlsx` /GSTR-1-CSV download, `requireGstFilingAccess()` gating — is unchanged since the Fourth Edition. `requireGstFilingAccess()` was also centralized into `src/lib/apiAuth.ts` (2026-09-16) — a pure dedup, not a behavior change.

9.12 Settings

Unchanged since the Fourth Edition.

9.13 Recycle Bin

UPDATE — cache-invalidation gap closed (2026-09-09) — no structural change to Bin's retention/purge model. What did change: restoring an invoice/purchase-bill/credit-note from the Bin re-applies a real stock effect but previously never busted `/api/products`, leaving product list/stock figures stale after a restore until a hard refresh; this is now fixed.

9.14–9.16 Admin, Global Search, Email

Unchanged since the Fourth Edition beyond what's covered elsewhere in this chapter (a fourth email-send route, `/api/send-statement`, is covered in Chapter 18.4).

9.17 PDF Generation

UPDATE — invoice PDF page-2+ header repeat rewritten (2026-09-09) — the Fourth Edition described the 2026-08-19 renderer rewrite as final. It wasn't quite: a multi-page invoice's continuation pages repeated the banner but never the item column-header row ("Qty"/"Rate"/"GST"/"Amount"), because that row lived inside `<tbody>` and the renderer only measured `<thead tr>` as a repeating region. The first attempted fix — moving the column-header into `<thead>` — introduced a worse regression: it jumped to directly under the banner on page 1, ahead of Invoice No./Bill To/Place of Supply. Caught by manual review, not an automated check. The corrected design: the column-header row stays a normal `<tbody>` row, measured as an independent second repeating region alongside the banner's own. A continuation page redraws the column-header only if it actually contains an item row, so a trailing Notes/Bank/Terms/Totals-only page gets just the banner. This two-region approach was applied only to the invoice print area.

UPDATE — JPEG→PNG page images, cache-first Print flow, bumped render version — `RENDER_VERSION` is now `2` and is baked into every cache key, so bumping it invalidates every previously-cached PDF app-wide. The invoice detail page's Print action was also made cache-first, reusing an already-cached PDF instead of unconditionally re-rendering on every click.

UPDATE — `withCachedPdf()` refactor — the cache-check/render/cache-save boilerplate, previously hand-duplicated across all 9 PDF-generating call sites, is now one shared exported helper. The render

strategy itself (DOM read vs. hidden-iframe) is untouched per caller.

9.18 Rate Lists

Re-confirmed unchanged in every respect the Fourth Edition documents.

9.19 Configurable Document Numbering & the Indian Financial Year

Unchanged, extended with the next-number preview mechanism described under 9.1.

9.20 Shared UI Building Blocks

NEW THIS EDITION — loading-skeleton fidelity audit (2026-09-10) — a dedicated 6-agent parallel audit compared every page/popup's loading skeleton against its real loaded markup. 13 real mismatches were found and fixed, all skeleton-only edits with no change to loaded UI: Settings, Invoice edit, Invoice detail, Rate List detail and edit, Customer/Product/Vendor edit, Purchase Bill detail, and Bin/Admin/Permissions. Dashboard, both Overview pages, every list page, both Statement pages, `TaxonomyDetailPage`, and every modal/popup checked were confirmed already in sync.

NEW THIS EDITION — useEntitySearch.ts (2026-09-10) — see 9.6 above.

9.21 "My Profile" (Self-Service)

NEW THIS EDITION — IN PROGRESS, NOT YET COMMITTED AS OF 2026-09-17 — see Chapter 6.7 for the full account. A dedicated app-wide manager-role permission audit confirmed the section-permission system itself is sound, but surfaced one real, previously-unnoticed gap: clicking your own avatar unconditionally linked to `/admin` regardless of role, and `/admin` immediately bounces any non-admin session to `/dashboard`. Since `GET / PUT /api/admin/profile` was already fully role-agnostic, this meant staff and manager accounts had no working UI at all to change their own name, login email, or password. The "My Profile" card was extracted into a new shared component, `src/components/profile/MyProfileCard.tsx`, and a new standalone page, `src/app/(dashboard)/profile/page.tsx`, renders it for every role. `admin/page.tsx` renders the same component so the admin's own equivalent section is unchanged in substance, just componentized.

9.22 Dashboard Financials / Cash Flow — the Weighted-Average-Cost Engine Story

Module-level account (see Chapters 1/26 for the narrative telling, Chapter 20/28 for the error-handling/lessons-learned angle). Over eight commits on 2026-09-15, an "Actual Profit" costing engine was built into the Dashboard's Financials/Cash Flow section: `src/lib/inventoryCosting.ts` performed a chronological weighted-average-cost (WAC) replay over each product's full `StockMovement` history to

attribute a real per-unit cost to every sale, alongside the new `PeriodFilter` component and new `InvoiceItem` / `ReturnItem` cost columns.

It was fully removed the next day, 2026-09-16, commit `eee89b8` ("Remove WAC/Actual Profit costing engine after oversell data-corruption bug"). A live QA pass found the engine silently mis-costed any oversold product while still labeling the resulting figure "verified"/ledger-sourced with no visible warning — the commit message records a concrete reproduction: a sale that physically came entirely from a ₹50/unit purchase batch was costed at ₹100/unit and marked "ledger". Because the defect was in the core replay algorithm itself, not a display detail, the entire feature was removed rather than patched — `src/lib/inventoryCosting.ts` is confirmed deleted, along with the cost columns, the Actual Profit dashboard section, the custom-item "Cost"/mandatory-Selling-Price UI it had introduced, and the cost-linking-specific parts of the return/duplicate-item validation added alongside it.

What survived the removal and is confirmed present in the current dashboard code: Cash Flow's simple total-sales/total-purchases arithmetic (never implicated in the bug) plus the GST Collected / GST Paid (ITC) / Net GST Payable tiles, and `PeriodFilter` as the dashboard's FY+month selector.

The same commit also centralized `requireGstFilingAccess()` into `apiAuth.ts` and folded in three unrelated fixes found during the same deep-QA pass: `Customer.creditLimit`'s "Infinity" rejection, the purchase-bill CGST/SGST footing fix, and the purchase-bill WebP fourcc validation.

10 Invoice Flow — Step by Step

The thirteen-step flow, the transportation-charge mechanics, the configurable number format, the FY-bounded editable date, and the editable "Bill To" customer are all unchanged in substance since the Fourth Edition. Two additions apply on top of that unchanged flow:

- **Next-number preview.** Before Step 1 even begins, the New Invoice page's title bar shows a live badge previewing the number Step 8's submission would receive if saved right now. This is read-only and reflects, but does not reserve, the outcome — if a second invoice is created by someone else in the gap, the actual assigned number can differ. The real number is still, and only ever, assigned inside `POST /api/invoices`'s own Serializable transaction with retry-on-conflict.
- **Sort order after creation.** Once saved, the new invoice's position on the Invoices list is governed by `createdAt`, not by the invoice's own `date` field — even if that `date` was backdated during entry, since `invoiceNumber` is always assigned in the same transaction the row is created in.

The idempotency mechanism at Step 8 and drag-to-reorder between Steps 3 and 6 are both unchanged and remain accurate as documented in the Fourth Edition.

11 Stock Flow & the Movement Ledger

Every rule in this chapter is **confirmed unchanged**, verified directly against `src/lib/stockMovement.ts`. The closed, specific `StockMovementType` union, the `batchAdjustStock()` batched-delta mechanism, the single shared low-stock definition, and the soft-delete-vs-permanent-delete asymmetry are all exactly as the Fourth Edition documented them.

MOVEMENT TYPE	DOCUMENTTYPE	WHEN IT HAPPENS
<code>sale</code>	<code>invoice</code>	A new invoice is created
<code>sale_edit_reverse</code> / <code>sale_edit_apply</code>	<code>invoice</code>	An invoice is edited — old quantities reversed, then new quantities applied
<code>sale_delete_restore</code>	<code>invoice</code>	An invoice is soft-deleted
<code>sale_bin_restore</code>	<code>invoice</code>	A deleted invoice is restored from the Recycle Bin
<code>purchase</code>	<code>purchase_bill</code>	A new purchase bill is created
<code>purchase_edit_reverse</code> / <code>purchase_edit_apply</code>	<code>purchase_bill</code>	A purchase bill's items are edited
<code>purchase_cancel</code> / <code>purchase_uncancel</code>	<code>purchase_bill</code>	A purchase bill is cancelled / un-cancelled
<code>purchase_delete_restore</code>	<code>purchase_bill</code>	A purchase bill is soft-deleted
<code>purchase_bin_restore</code>	<code>purchase_bill</code>	A deleted purchase bill is restored
<code>return</code>	<code>credit_note</code>	A credit note is issued against an invoice
<code>return_delete_reverse</code>	<code>credit_note</code>	A credit note is soft-deleted
<code>return_bin_restore</code>	<code>credit_note</code>	A deleted credit note is restored
<code>manual</code>	<code>manual</code>	A manual stock adjustment after a physical count

A whole costing engine was wired into this exact function and then cleanly reverted, with zero residue. Between the two editions, `batchAdjustStock()` briefly gained a follow-up call into `recostProducts()` to maintain a per-product weighted-average cost. It was fully removed the day after a live data-corruption bug was found (see 9.9/9.22). Reading `stockMovement.ts` today shows exactly the same function shape as the Fourth Edition documented — the `createMany()` call is the function's last action, with nothing downstream of it.

12 GST Logic

The core per-line formula, the CGST+SGST vs. IGST decision, discount treatment, the Reverse Charge disclosure-only flag, commercial rounding, the worked example, credit-note GST-rate preservation, and the numbering-as-compliance-control reasoning are all confirmed unchanged and still accurate — no GST rate-derivation or core rounding rule changed in this window. Two real, adjacent changes did land, both closing genuine under/over-reporting bugs rather than altering how tax is computed on any individual line:

- **Transport-charge GST is now folded into `salesRegisterByRate` and GST Filing's output-tax totals** (2026-09-09) — see 9.11 above for the full mechanism.
- **CGST/SGST rounding now foots to the printed Grand Total, on both Invoices and (this window) Purchase Bills.** `splitGstForDisplay()` rounds the combined total once and gives SGST whatever remains after CGST's own rounding, so the two printed figures always sum to exactly the printed total. Fixed on the invoice side first (2026-09-09); this window's Purchase Bill work extended the identical fix to the purchase-bill detail page and PDF.

The other adjacent change already documented in the Fourth Edition — purchase bills' duplicate-product-line guard — is unchanged and affects only which rows can exist on a bill, not how tax is computed on any individual row.

13 API Documentation (Full Reference)

Every endpoint below was re-confirmed against the current route handlers as of 2026-09-17. "Auth" shows the minimum permission guard applied (Session / Write / Admin / Section:<name> / GstFiling — the last being `requireGstFilingAccess()`, admin or both `reports_sales` + `reports_purchases`). All of `/api/**` additionally sits behind `middleware.ts`'s default-deny baseline.

This edition's changes to Chapter 13 are almost entirely additive — no route documented in the Fourth Edition was removed, and the weighted-average-cost costing engine that briefly touched several route files added zero new route files of its own; it only edited handler bodies, and the revert restored their pre-engine shape.

13.1 Sales

Unchanged from the Fourth Edition for the bulk of this group. New/changed since:

METHOD / PATH	AUTH	PURPOSE
<code>GET /api/invoices/next-number</code>	Session	New (2026-09-17). Read-only preview of the invoice number a save right now would get. Runs the identical <code>computeNextNumber()</code> / <code>numberFormatDbFilter()</code> logic the real create route uses, without the transaction/insert/override-consumption — a preview, not a reservation.
<code>GET /api/credit-notes/next-number?date=</code>	Session	New (2026-09-17). Same preview, for the credit-note sequence; takes the return's own picked date since the FY segment derives from it.
<code>DELETE /api/invoices/[id]/payment/[paymentId]</code>	Write	New (2026-09-07). The sibling <code>PUT</code> already existed but had no UI wired to it anywhere. Recomputes <code>paidAmount</code> /status inside a Serializable transaction (P2034 retry), and re-verifies the resulting <code>paidAmount</code> doesn't drop below the total already refunded via non-deleted credit notes, 400ing with a clear message if it would. Blocked entirely if the parent invoice is soft-deleted.
<code>GET/POST /api/customers</code> , <code>/api/vendors</code> , <code>/api/rate-lists</code>	Session / Write	Behavior changed (2026-09-06). All three POST create routes now accept an optional client-generated <code>idempotencyKey</code> — a retried/double-clicked submission returns the already-created row instead of creating a duplicate; a genuine concurrent race is resolved via a P2002 catch-and-refetch fallback.

13.2 Purchases

Unchanged from the Fourth Edition for the bulk of this group. New/changed since:

METHOD / PATH	AUTH	PURPOSE
GET /api/purchase-bills/next-number?billDate=	Session	New (2026-09-17). Same preview pattern, keyed on the bill's own picked <code>billDate</code> .
PUT /api/purchase-bills/[id]/payment/[paymentId]	Write	New (2026-09-07). This side had no edit route at all before. Mirrors the invoice payment PUT. No credit-note-floor guard needed here (no <code>PurchaseReturn</code> model), but a cancelled bill's status is explicitly preserved verbatim across the recompute.
DELETE /api/purchase-bills/[id]/payment/[paymentId]	Write	New (2026-09-07), same commit as the PUT above — likewise had no prior equivalent.
GET /api/purchase-bills/[id]/attachment/[filename]	Session	New, previously undocumented (added 2026-09-08). Streams the bill's own saved attachment through the server using the private Blob token, rather than a public URL. Fails fast with 503 if no private token is configured.
GET /api/purchase-bills/attachment/[folder]/[filename]	Session	New, same migration. Serves a just-uploaded, not-yet-saved attachment by its raw private-store pathname. Both path segments are regex-validated against the exact character set the upload route sanitizes filenames to.

13.3 Catalogue

Unchanged endpoint set from the Fourth Edition. No new Catalogue route surfaced in this window.

13.4 Reports & Filing

Unchanged from the Fourth Edition for the bulk of this group. New/changed since:

METHOD / PATH	AUTH	PURPOSE
GET /api/reports/eway-bill?from=&to=	GstFiling	New, previously undocumented despite existing since 2026-09-04 — omitted from the 4th-edition-era table by oversight. Lists every non-deleted invoice/purchase bill at or above <code>EWAY_BILL_THRESHOLD</code> , for the optional IST date range — the raw data feed behind the E-way Bill Reports page.
GET /api/reports?type=financial-years	Session	New (2026-09-15, kept after the same-window costing-engine revert). Returns the list of financial years that have any invoice/purchase-bill data, backing the Dashboard's FY period-selector dropdown.
GET /api/reports?type=net-gst-payable&fy=&month=	GstFiling	New (2026-09-16, added when the costing engine's flawed "Actual Profit" section was removed and Cash Flow was given a real tax-liability tile in its place). Reuses <code>buildGstFilingReport()</code> directly.

METHOD / PATH	AUTH	PURPOSE
<code>GET /api/reports?type=combined-dashboard&fy=&month=</code>	Session (section-filtered)	Behavior changed: now accepts <code>?fy= / ?month=</code> period params, threaded into a <code>getDashboardFinancials()</code> call whose result is merged into the response under a <code>financials</code> key.

13.5 System & Admin

Unchanged endpoint set from the Fourth Edition, re-confirmed — no new System/Admin route in this window. `requireGstFilingAccess()` itself was centralized into `src/lib/apiAuth.ts` on 2026-09-16 (previously duplicated between `gst-filing/route.ts` and `reports/eway-bill/route.ts`) — an internal refactor with no change to any route's external auth behavior.

UPDATE — pagination pairing count: the list-route/stats-route pairing remains unchanged at the same set of entities documented in the Fourth Edition — none of this window's additions are list routes, so none needed or gained a `/stats` companion.

14 Frontend Architecture

14.1 Pages & layouts

Unchanged since the Fourth Edition, re-confirmed — every authenticated page still sits under `DashboardShell`, and a new sibling was added rather than anything existing being restructured: `src/app/(dashboard)/profile/page.tsx` (2026-09-17) gives staff/manager a working "My Profile" page. It renders a new shared `MyProfileCard` component that `admin/page.tsx` now also renders, in place of the ~120 lines of inline markup that page used to own directly.

14.2 State management

Still no Redux/Zustand — `useFetch(url)` backed by the shared in-memory cache remains the entire data layer for lists/details. New this edition, one layer below: `src/lib/useEntitySearch.ts` (2026-09-10) replaces a client-state pattern that had nothing to do with `useFetch`'s own cache — the vendor picker on Purchase Bill and the customer picker on Invoice used to fetch the *entire* table once (`?pageSize=5000`, clamped to 2000 server-side) into local state and filter it client-side per keystroke. That silently stopped showing anything past whichever cap won once a business's real vendor/customer count grew past it — a correctness bug, not just a performance one.

```
export function useEntitySearch<T>(endpoint: string, query: string, opts?: { pageSize?: number; enabled?: boolean }) {
  const pageSize = opts?.pageSize ?? 20;
  const debounced = useDebounceValue(query, 250);
  useEffect(() => {
    // fetch(endpoint + "?" + params, { signal, headers: { "x-no-loader": "1" } })
    // AbortController cancels the in-flight request on a newer keystroke
  }, [endpoint, debounced, pageSize]);
  return { results, total, loading };
}
```

It debounces the typed query 250ms, issues a live `search`-param request against the entity's own already-existing paginated list route, and cancels the in-flight request the instant a newer keystroke supersedes it — the identical shape `GlobalSearch.tsx` already used for the topbar search, just extracted into a reusable hook. It deliberately bypasses `useFetch`'s shared cache since a picker's live keystroke-driven query string would otherwise pollute that cache with one entry per keystroke. The picker's *selected* vendor/customer is now tracked as its own resolved object instead of a `.find()` lookup over the old prefetched array.

14.3 Forms & validation

Unchanged since the Fourth Edition, re-confirmed.

14.4 The app-wide Modal component

Unchanged — `variant="fullscreen"` remains the only variant any call site actually uses. The new `BulkDownloadOptionsDialog` deliberately does not use `Modal` — it mirrors `PdfCopyDialog`'s smaller, non-fullscreen popup shape instead, since it's a two-checkbox choice, not a form.

14.5 Shared UI components

Unchanged in the main table from the Fourth Edition, with two additions: `useEntitySearch` (shared debounced/cancellable live-search hook, 2026-09-10) and `BulkDownloadOptionsDialog` (a small `PdfCopyDialog`-shaped popup, 2026-09-17). The app-wide loading-skeleton fidelity audit (13 real mismatches found and fixed, 2026-09-10) is covered in full in Chapter 9.20.

14.6–14.8 Rate List forms, Pagination and mobile breakpoints, Theming

Unchanged since the Fourth Edition, re-confirmed — the three-breakpoint system plus the sub-480px topbar `MoreMenu` condensation are all unchanged.

15 Backend Architecture

The "no controller/service/repository layering" architecture is unchanged. `src/lib/` gained several genuinely new modules since the Fourth Edition, and lost one that was built and then fully reverted in the same window:

HELPER MODULE	RESPONSIBILITY
<code>attachmentHref.ts</code>	New (2026-09-08). The one place either purchase-bill-attachment URL shape gets built — <code>purchaseBillAttachmentByIdHref(billId, filename)</code> for a saved bill, <code>purchaseBillAttachmentPendingHref(blobUrl)</code> for a just-uploaded replacement. Both deliberately end the URL in the real filename, not a fixed route segment.
<code>bulkPdfDownload.ts</code>	New (2026-09-17). <code>bulkZipDownload()</code> — the generic form, where each item may contribute zero, one, or several named entries — and <code>bulkDownloadPdfsAsZip()</code> built on top of it. Dynamically imports <code>jszip</code> .
<code>validationCsv.ts</code>	New (2026-09-05). <code>buildValidationCsv(issues, noIssuesMessage)</code> — the single shared "Validation-Report.csv" builder, previously copy-pasted once inside <code>gstFilingZip.ts</code> and once inside <code>gstr1CsvZip.ts</code> .

One module from this edition's own arc is gone entirely: `src/lib/inventoryCosting.ts` was built on 2026-09-15 as the core of the weighted-average-cost "Actual Profit" engine, then fully deleted the next day after a live QA pass found its chronological-replay algorithm silently mis-costed any oversold product while still labeling the wrong result as "verified." Do not list it as a current helper module, and do not re-attempt a costing engine without first reading `DEVELOPMENT_LOG.md` Era 20 in full.

`pdfCache.ts` and `useIdempotencyKey.ts` are both unchanged in their original external contracts this edition; `pdfCache.ts` gained one new exported function (`withCachedPdf()`) — see Chapter 17.

15.1 Per-entity query builders

Unchanged — the same set of `xQuery.ts` modules. File uploads remain magic-byte validated, with one specific gap closed this edition: `.webp` upload validation now also checks the WEBP fourcc at file offset 8-11, not just the RIFF container bytes — closing a spoofable-file-type gap. See Chapter 19.

16 Data Flow

The overall request/response/cache diagram is unchanged. Every mutation still calls `revalidateTag(tag, { expire: 0 })` and writes an Activity Log entry.

16.1–16.2 bustCache vs. bustCachePrefix, Idempotency keys

Both unchanged since the Fourth Edition.

16.3 The same bug class recurred at more call sites — closed in one audit pass, 2026-09-09

UPDATE — continuation of 16.1's bug class. This is not a new bug — it is the same class of gap 16.1 already found and partially fixed (a mutation busting a base resource's cache from outside that resource's own page, and missing a sibling sub-route or a related resource entirely), recurring at several more call sites a dedicated multi-agent audit surfaced and closed in one pass: PDF-cache invalidation gaps on the invoice detail page's payment/return/delete handlers, and both statement pages' PDF cache key missing `settings.updatedAt` /the party's own `updatedAt` ; app-wide `bustCache` / `bustCachePrefix` gaps across invoice/purchase-bill list-page deletes, vendor edit, product delete/adjust-stock, brand/category detail-page rename, Bin restores, and the inline Add/Edit Customer modals. No change to `bustCache()` / `bustCachePrefix()` themselves was needed this time — every fix here was a missing call site, not a broken primitive.

16.4 A related, distinct hardening pass — cache-check/render/cache-save boilerplate unified

`pdfCache.ts` gained a new exported helper, `withCachedPdf(entity, entityId, variantKey, render, force?)`, in the same 2026-09-09 pass — see Chapter 17.4.

17 PDF Generation

The overall pipeline is unchanged in concept, and the Fourth Edition's account of the 2026-08-19 renderer rewrite is unchanged and still accurate.

UPDATE — PDF pipeline polish, 2026-09-09. Per-page canvas export switched from JPEG to PNG, and jsPDF's "SLOW" compression flag was added — previously omitted entirely, embedding each page's image fully uncompressed at roughly 10MB per page. The invoice detail page's Print button stopped force-regenerating the PDF on every click and switched onto the same cache-first `withCachedPdf()` path View/Download/Share already used.

17.2 Long-document page splitting — reworked this edition

UPDATE — two independently-measured repeating regions, 2026-09-09. A real, previously-unfixed rendering bug: a multi-page invoice's continuation pages repeated only the banner — never the item column-header row — because that row lived inside `<tbody>`, and the page-splitting logic only ever measured `<thead>` for what to repeat. Simply moving the column-header row into `<thead>` would have fixed the repeat but introduced a worse regression: it would jump to directly under the banner on page 1, ahead of Invoice No./Bill To/Place of Supply. The shipped fix: the column-header row stays a normal `<tbody>` row, marked `data-header-item-cols="true"`, and is measured as its own independent second repeating region alongside the banner's pre-existing region. A continuation page redraws the column-header only if it actually contains at least one item row on it. This two-region approach was applied only to the invoice print area — Purchase Bill's own `<thead>` has no separate banner, and the Credit Note print area's three-row `<thead>` was already in correct document order.

`pdfCache.ts`'s `RENDER_VERSION` constant was bumped from 1 to 2 in the same window specifically because of this change — without the bump, a browser could keep serving old, header-less continuation-page output indefinitely.

17.4 `pdfCache.ts` — the client-side IndexedDB PDF cache

NEW THIS EDITION — `withCachedPdf()`. Every PDF-generating page (9 call sites total) had been hand-duplicating an "if not forced, check cache; else render and save to cache" boilerplate.

`withCachedPdf()` unifies only the caching boilerplate itself — the render strategy stays per-page, since it genuinely differs by data availability.

18 Email System

The six-step flow is unchanged. `src/app/api/send-invoice/route.ts` has no commits against it since the Fourth Edition.

18.1–18.3 `/api/send-invoice`, `/api/send-rate-list`, `/api/send-purchase-bill`

All unchanged since their respective prior documentation.

18.4 `/api/send-statement` — new since the Fourth Edition

NEW THIS EDITION — added 2026-09-04. A fourth PDF-email sender, added alongside Customer/Vendor account statements. Structurally the closest sibling to `/api/send-purchase-bill` : the same comma-separated, up-to-10-recipient `to` field, the same rate limit, the same explicit SMTP timeouts, the same 10MB attachment cap, and the same header-injection guard. The one genuine structural difference: a statement has no fixed document number, so the email's subject and PDF filename are built from a free-text title field instead.

19 Security

19.1 Authentication security

Unchanged base mechanisms. New since the Fourth Edition:

UPDATE — self-password-change rate limiting, IN PROGRESS, NOT YET COMMITTED AS OF 2026-09-17. Verified directly via `git diff HEAD` on both files: `PUT /api/admin/profile` and the self-edit path of `PUT /api/admin/users/[id]` each gained a `rateLimit("profile-password:<userId>", 10, 15 * 60 * 1000)` check before validating `currentPassword`, returning 429 once exceeded. Reasoning taken directly from the code's own comment: "A session/cookie thief who doesn't know the real password could otherwise brute-force `currentPassword` here with no login-page throttle to stop them" — closing a gap where the login page's own rate limits didn't extend to this second path to the same password hash.

FIXED — AUTH-001 / AUTH-002, 2026-08-19 remain in place unchanged, re-confirmed.

19.2 Authorization

Unchanged — see the manager-role audit in Chapter 6.6, which re-confirmed the four guard functions still hold under a fresh, real seeded manager account rather than by code review alone.

19.3 Input validation / injection protection

Unchanged base protections. Two items closed since the Fourth Edition, both already committed in `eee89b8` (2026-09-16, the WAC-engine-removal commit):

FIXED — Customer.creditLimit accepting non-finite input. `validateCreditLimitInput()` previously had no finiteness check — a submitted literal `"Infinity"` parses to a valid JS number via `parseFloat`, and since the app's own contract for a blank credit limit is "no limit," a non-finite value silently reached storage as an effectively-unlimited credit line. Confirmed current: the function now rejects any value outside `[0, MAX_MONEY_VALUE]` or non-finite.

FIXED — purchase-bill .webp upload checked only the RIFF container, not the WEBP fourcc. The magic-byte check previously verified only the four-byte RIFF container signature — a signature shared by every RIFF-family file — so a non-image RIFF file renamed to `.webp` could pass. Confirmed current: it now also checks the WEBP fourcc at offset 8-11.

UPDATE — same fix extended to the logo upload route, plus a new HEIC brand check, IN PROGRESS, NOT YET COMMITTED AS OF 2026-09-17. `eee89b8` only touched the purchase-bill attachment route. A `git diff HEAD` on `src/app/api/settings/logo/route.ts` shows the identical gap being closed there too. The same uncommitted diff also hardens the HEIC check on the purchase-bill

upload route: it previously confirmed only the generic ISO-BMFF `ftyp` container marker, shared by every ISO base-media-format file — it now additionally reads the 4-character major-brand field and checks it against the real-world HEIC/HEIF brand set.

19.4 API trust-boundary fixes

Unchanged from the Fourth Edition's API-001/002/003. One residual gap in that same family is being closed this edition:

UPDATE — credit-limit TOCTOU gap on invoice edit, IN PROGRESS, NOT YET

COMMITTED AS OF 2026-09-17. `CLAUDE.md` 's Features Completed #44 states the customer credit-limit check "runs inside the same Serializable transaction as the invoice create/edit" — true for create, but a direct read shows this was not actually true for edit until now. `invoices/route.ts` (create) has carried `Serializable` isolation with a 5-attempt P2034 retry loop since Era 15. `invoices/[id]/route.ts` (edit), by contrast, ran the credit check inside a plain transaction with no explicit isolation level — Read Committed, which does not detect the write-skew anomaly two concurrent edits could produce. The working-tree diff brings edit up to the same guarantee create has had. Until committed, this is a description of intent sitting in the working tree, not of what's deployed.

19.5 Blob-storage & asset validation

Unchanged from the Fourth Edition. Era 19.5 (2026-09-08) separately moved purchase-bill attachments to a private Vercel Blob store served only through an authenticated proxy route — described fully under Purchase Bills' module documentation (Chapter 9.7) rather than repeated here.

19.6–19.7 Secrets at rest, Rate limiting

Both unchanged. The new `profile-password:<userId>` limiter (19.1) uses the identical underlying `rateLimit()` helper and inherits the same per-instance, resets-on-redeploy caveat as every other rate-limited route.

19.8 CSV/XLSX formula-injection hardening

Unchanged from the Fourth Edition — `neutralizeFormulaCell()` is committed and further consolidated during Era 15's audit into the shared `src/lib/validationCsv.ts` builder — closing the risk that the GSTR-1 CSV export had shipped without ever being routed through the same guard.

19.9 First-time setup safety, default-deny middleware

Unchanged.

20 Error Handling

The frontend banner-vs-toast distinction and the backend status-code table are unchanged.

Permanent close-out of the IST/UTC date-boundary bug class (2026-09-09), plus a pre-commit guard

`scripts/check-date-safety.js` greps staged files — or, via `node scripts/check-date-safety.js --all`, the whole `src/` tree — for two anti-patterns that have each independently caused real, shipped bugs in this app: (1) a `.toLocaleDateString()` / `.toLocaleString()` / `.toLocaleTimeString()` call with day/month/year/hour options and no explicit `timeZone`, outside `src/lib/formatDate.ts`; and (2) a bare `new Date(x)` parse of a plain date-only string inside an API route, which parses as UTC midnight rather than IST midnight. It is wired into the existing pre-commit git hook, so a commit containing either anti-pattern is blocked with a file:line list before it can land, rather than relying on a reviewer to catch it a third or fourth time. One small, cosmetic drift worth flagging: the script's own header comment still lists `formatCreatedSub()` as one of the approved display helpers — that function was deliberately built, then deliberately deleted again the same window per `CLAUDE.md`'s Features Completed #56. The comment is stale but harmless.

Optimistic-concurrency conflicts

Unchanged mechanism. See Chapter 19.4 above for the one place this window extends it further — invoice edit's credit-limit check gaining the same Serializable-isolation-plus-retry treatment invoice create already had, still uncommitted as of this edition.

Edit Payment's missing dirty-check, and unguarded `fetch()` calls on the invoice detail page — FIXED, 2026-09-07

A genuine, user-caught UX/robustness gap in the payment-edit/delete feature that had otherwise already passed a full live-verification pass: live-verification had exercised the happy path and the guard-rejected path, but not the idle path or the transport-failure path. Two real issues surfaced when the user asked directly why the Edit Payment modal's Save button stayed enabled with nothing changed:

- **No dirty-check on Save.** Fixed with the same `useDirty()` hook Rate List Edit already used — `openEditPayment()` calls `markClean()` the instant the modal opens; Save is disabled when `!isDirty`; the submit handler itself also short-circuits as defense-in-depth.
- **No `try/catch` around three invoice-detail-page payment `fetch()` calls.** A dropped connection mid-save would leave the saving/deleting flag stuck `true` permanently: the `OverlayLoader` never clears and the button stays disabled with no recovery short of a full page reload. Traced back to `handleAddPayment` specifically — a pre-existing issue carried forward when the new handlers copied its shape.

Both fixes were live-verified via Playwright against the real dev database — the network-error fix by actually aborting the in-flight request and confirming a "Network error" toast appears, rather than asserting the fix only from reading the code. Worth restating as a durable lesson: a live-verification pass that only exercises the happy path and the guard-rejected path will still miss the idle and transport-failure paths — each needs its own explicit check.

Database, activity logging

Unchanged.

21 Deployment

The development and production flows are unchanged in shape (push to `main` → Vercel builds → migrations apply → deploy). Two changes since the Fourth Edition, one a script change and one a real production incident:

UPDATE — `prebuild` now wipes the whole `.next` folder, not just `.next/dev`, 2026-09-06.

The Fourth Edition documented `prebuild` as deleting only `.next/dev` before every build. This changed to a full-folder wipe (commit `8f78b91`). Per `CLAUDE.md`'s Known Issues section, this was a second, separate build-reliability fix found while stress-testing `npm run build`: re-running `next build` against an already-built `.next` folder intermittently corrupted `.next/dev/types/validator.ts` mid-generation — a Turbopack concurrent-write race, reproduced directly (roughly half of repeated builds against an existing `.next` failed) and not fixable by editing generated output, only avoided by never building on top of a previous one. Since Vercel caches `.next` between deployments by default, this is flagged as a plausible cause of intermittent production build failures too — the trade-off (every build is now a cold build) is accepted deliberately for reliability.

FIXED — same-day production outage from a Vercel env-var rename, 2026-09-08. The private-Blob-store migration itself shipped cleanly, but its same-day follow-up commit introduced separate Production/Preview env-var names for the private store's token, since Vercel refuses to let two Blob store connections in one project share the same auto-injected variable name. Vercel's own "Connect to Project" flow renamed the Production-scoped token from `PRIVATE_BLOB_READ_WRITE_TOKEN` to `LIVE_PRIVATE_BLOB_READ_WRITE_TOKEN`, but `getPrivateBlobToken()`'s fallback chain still checked the old name first — so production silently fell through to the dev store's token instead, and every purchase-bill attachment fetch returned 403 Forbidden live in production. Fixed by reordering the chain to `LIVE... → DEV... → the unprefixed PRIVATE_BLOB_READ_WRITE_TOKEN` (the last only ever populated locally). The practical lesson recorded in both `CLAUDE.md` and `DEVELOPMENT_LOG.md`: if a "403 on every attachment" report ever recurs, check this fallback chain against Vercel's current Storage-tab env-var names first.

The retrying migrate-deploy wrapper is otherwise unchanged. The `postinstall` script is unchanged and remains mandatory. The pre-commit hook it installs now also runs the new `check-date-safety.js` guard alongside the pre-existing schema-migration guard.

22 Third-Party Services

The service table (Vercel, Neon, Vercel Blob, Gmail SMTP via Nodemailer, Razorpay's public IFSC directory) is unchanged in the count of distinct third-party services — five, still.

UPDATE THIS EDITION — Vercel Blob is now two stores, not a new service. The private-access purchase-bill-attachment store introduced 2026-09-08 is a second store provisioned within the same Vercel Blob product/account this app already used for the business logo — billed, provisioned, and administered through the identical Vercel Storage dashboard, just with its own access mode and its own read-write token(s). Recorded here as an access-pattern and configuration change to an existing third-party dependency, not as a sixth service added to the table.

Purchase-bill emails and customer/vendor statement emails remain clients of the Gmail SMTP integration alongside invoices and rate lists — unchanged.

23 Environment Variables

The Fourth Edition's list is superseded on the Blob side this edition:

VARIABLE	REQUIRED	NOTES
<code>DATABASE_URL</code>	Yes	Unchanged.
<code>NEXTAUTH_SECRET</code>	Yes	Unchanged.
<code>NEXTAUTH_URL</code>	Production	Unchanged — see Chapter 21's Known Issues history.
<code>ENCRYPTION_KEY</code>	Optional	Unchanged.
<code>EMAIL_USER</code> / <code>EMAIL_APP_PASSWORD</code>	For email sending	Unchanged.
<code>BLOB_READ_WRITE_TOKEN</code>	For the business logo only	Narrowed scope this edition — as of the Fourth Edition this token backed all Blob storage. Since the 2026-09-08 migration it authorizes only the original public store, which now holds the business logo exclusively.
<code>PRIVATE_BLOB_READ_WRITE_TOKEN</code> / <code>LIVE_PRIVATE_BLOB_READ_WRITE_TOKEN</code> / <code>DEV_PRIVATE_BLOB_READ_WRITE_TOKEN</code>	For purchase-bill attachments	New this edition — authorizes the second, private-access Blob store. <code>getPrivateBlobToken()</code> checks, in order: <code>LIVE...</code> (Production), <code>DEV...</code> (Preview), then the unprefixed variable (local development only). This three-way chain exists purely because Vercel auto-injects a different variable name per environment once a second Blob store is connected to one project. Without any of the three set, attachment upload and every attachment-fetch route return a clear 503 rather than silently falling through to the wrong store's token.

The distinction to hold onto: `BLOB_READ_WRITE_TOKEN` and the `*PRIVATE_BLOB_READ_WRITE_TOKEN` family now point at two different Blob stores with two different access modes — treating them as interchangeable will not work and will not error clearly at write time, only at read time, as the 2026-09-08 outage in Chapter 21 demonstrated.

24 Project Dependencies

The overall dependency list is unchanged since the Fourth Edition — zero dependency version changes, and no new package added. What changed is scripts only:

CHANGE	DETAIL
<code>prebuild</code>	Widened from deleting only <code>.next/dev</code> to deleting the whole <code>.next</code> folder (2026-09-06) — see Chapter 21.
<code>check:dates</code> (new script)	<code>node scripts/check-date-safety.js --all</code> — runs the new pre-commit date-safety guard manually over the whole tree.

No new runtime package was added for any feature shipped in this window, including the ones that might plausibly have needed one: month-wise bulk PDF ZIP download is built on JSZip, unchanged at `^3.10.1` — already a dependency as of the Fourth Edition, just gained a second internal caller. The private-Blob-store migration is built on `@vercel/blob`, unchanged at `^2.5.0`. `next` remains `^16.3.1` and `nodemailer` remains `^9.0.5`. Every other dependency in the Fourth Edition's table is unchanged in this window.

25 Business Workflow — A Day in the Life

The narrative established across prior editions — morning dashboard check, a purchase bill arriving, a Rate List being built and emailed to a prospect, that prospect becoming an invoice, payment recorded, a credit note issued, month-end vendor payment, GST filing at tax time, the Recycle Bin as a safety net — is unchanged in every particular. Three further moments are worth narrating explicitly this edition, each grounded in a feature verified directly against source:

1. At the end of the month, instead of opening thirty-odd invoices one at a time to download each PDF for the accountant, the office staff picks the month and year on the Invoices list page and clicks "Bulk Download (ZIP)" — the app renders every matching invoice's PDF in turn and hands back one ZIP file. Doing the same on the Purchase Bills page offers a choice first: the generated bill PDFs, the vendors' own uploaded scans, or both — since a vendor's bill scan is only ever fetched through the app's own authenticated route, never a bare link, this is also the first time that stored file can leave the system at all outside of viewing it in-browser one bill at a time.
2. A staff member realises, a day after recording it, that a customer's payment was typed in as ₹15,000 instead of ₹1,500 — a fat-fingered extra zero. Previously this had no clean fix at all; now they open the invoice, find the payment in its history table, and edit the amount directly. If that correction would leave a previously-issued credit note claiming more refunded cash than the invoice ever actually received, the app refuses the edit outright with a clear message instead of quietly letting the books go out of balance.
3. A day-to-day billing clerk (a staff account, not an admin) wants to change their own login password after a routine security reminder. Instead of asking an admin to do it for them — the only real option beforehand, since the page that could do this lived behind an admin-only wall — they click their own name in the sidebar and land on a "My Profile" page built exactly for this, and change it themselves. (This capability is implemented and verified but not yet committed to the codebase as of 2026-09-17.)

26 Non-Technical Explanation

All the framings established in prior editions — the shop owner's smart notebook, the risk-minded owner's safety net, the school student's library analogy, the "never written a line of code" explanation of what a button click actually does, and the app now noticing and refreshing related screens on its own — are unchanged and remain accurate descriptions of how the system behaves today.

One addition is worth explaining in the same plain terms, because it is the single most conceptually important thing that happened in this window, and it is a story about restraint, not failure.

The team tried to teach the app something genuinely hard: for every single sale, work out not just what it was sold for, but what it actually cost the business to have that item on the shelf in the first place — averaged fairly across however many different purchase batches, at however many different prices, had gone into that stock over time — so that the Dashboard could show a real, trustworthy profit figure instead of just "money in" versus "money out". This is a genuinely useful thing for a business to know, and it was built, in full, over the course of a single day.

The very next day, before it was trusted with real customers' numbers, someone checked its answers against the real data already sitting in the system — and found a case where it was wrong. Specifically: this app has always deliberately allowed a member of staff to sell more units of something than are technically shown as "in stock" at that exact moment (a common, sensible real-world allowance — the stock count catches up once the matching purchase is entered). The profit-tracking feature had no correct way to work out the cost of those particular units, because by its own accounting there weren't enough of them yet — so it made a guess. That alone might have been forgivable. What made it unsafe was that the guess was then labelled exactly the same way as a properly-computed, fully-verified answer, with no way for anyone looking at the Dashboard to tell the two apart.

A wrong number that looks exactly like a right number is worse than no number at all, because it invites a business decision to be made on top of it. So rather than patch the one bad case and hope no similar one existed, the whole feature was removed the same day it was found — cleanly, down to its database columns — and the team went back to a simpler, honest comparison of total sales against total purchases while a correct version is designed properly. The parts of that day's work that were independently correct and useful on their own — a set of real GST figures on the Dashboard, and a way to pick "which month/financial year am I looking at" — were kept, because they didn't share the flawed part's problem. Nothing about this was a failure of the app; it is exactly what a careful business system is supposed to do when it discovers it might be about to tell its owner something untrue.

27 Developer Guide

Sections 27.1 through 27.7 (first-time setup, first admin account, making a schema change, adding a new page, adding a new API route, adding a new paginated list endpoint, adding a Rate-List-adjacent bulk-entry feature) are unchanged from the Fourth Edition and remain the correct procedures.

27.8 Rules that must never be broken — updated this edition

Every rule from the Fourth Edition's equivalent list remains true, including the three rules introduced there (`bustCachePrefix()` sibling-sub-path busting, the client-generated idempotency-key pattern for retryable creates, and checking `r.ok` before treating a single-entity fetch's body as real data). Six further conventions have hardened into hard rules since 2026-08-31, all drawn directly from `DEVELOPMENT_LOG.md`'s own Conventions section, established the hard way across Eras 15 through 21 (2026-09-05 through 2026-09-17):

1. **Any edit-form Save button needs a dirty-check gate (`useDirty()`), not just a `saving` gate.** Without one, clicking Save with nothing changed still re-submits — an unneeded write and, on a route that logs one, a spurious `ActivityLog` entry. First established for Rate List Edit, it was missed on the new Edit Payment modals until the user's own review caught it.
2. **Every `fetch()` call inside an async save/delete handler needs its own `try/catch` , even when the route itself is normally reliable.** A thrown network-level rejection skips straight past an unwrapped `if (res.ok)` check and leaves whatever `saving / deleting` flag gates the UI stuck `true` forever.
3. **A point-in-time cap checked only at creation must be re-verified whenever the value it was capped against can later change.** A credit note is capped against an invoice's `paidAmount` only at the credit note's own creation time; editing or deleting that payment afterward doesn't re-run the check, so the re-check has to live in the payment's own edit/delete path instead.
4. **A return/credit-note line must inherit its per-line values from the traceable parent document line, never accept a client-supplied override, whenever a `productId` links the two.** Established early for `gstRate` and extended to `discountPercent` — closing a real over-crediting bug where a return against a discounted invoice line had been computing its taxable value at the pre-discount gross price.
5. **The big one: a chronological-replay costing/valuation algorithm must explicitly handle this app's own deliberate oversell behavior, and must never label a fallback/guessed value as "verified" or "ledger"-sourced.** A full weighted-average-cost "Actual Profit" engine was built and then fully removed the very next day after a live QA pass found it silently mis-costing units sold from an oversold product while the result still carried a "verified" label. The bug lived in the core replay algorithm, not a display detail, which is why the whole feature was pulled rather than patched. Do not re-attempt a costing/profit feature without reading `DEVELOPMENT_LOG.md` Era 20 in full and fixing this specific gap first.
6. **Process-level, not just documentation: the recurring IST/UTC date-boundary bug class now has an automated pre-commit guard, not merely a written warning.** `scripts/check-date-safety.js` blocks a commit outright rather than trusting a developer (or an agent) to remember.

It was wired in after this exact bug class had already resurfaced three separate times across earlier "fixes" — the lesson generalizes beyond dates: a bug proven to recur despite documentation is a candidate for an automated guard, not another round of the same written reminder.

28 Future Improvements

This chapter, as in every prior edition, reflects the project's actual, current Pending Tasks and Known Issues — taken directly from `CLAUDE.md` as of 2026-09-17 — not a fresh audit's speculative wishlist.

Carried over and still genuinely open

- Verify the email send flow end-to-end in a real environment. Unchanged since the Fourth Edition.
- Test bin restore/permanent-delete and empty-bin for all 8 entity types end-to-end. Still open.
- Multi-user simultaneous-edit notifications — still deferred pending a design decision on a real-time channel the app doesn't currently have.
- Scheduled/automatic reminder emails for overdue invoices, building on the email infrastructure that now spans four send routes.
- AI & automation opportunities (still open, unchanged): automatic GST-rate/HSN suggestions on new products, and dashboard anomaly flags.

Still open, re-confirmed this edition

- **Provision a real test database.** `tests/api/**` and `tests/e2e/**` are still written and pass type-checking/lint, but have never actually executed against a live database. Every substantial fix window since the Third Edition, including this edition's own bulk-download and manager-permission-audit work, has shipped without new automated regression tests for this same underlying reason, relying instead on live Playwright walkthroughs against seeded-and-cleaned-up rows in the real dev database.
- Test coverage remains genuinely incomplete in the same specific places `CLAUDE.md` itself lists.
- Rate Lists' one remaining deliberately-deferred gap: a public, token-based share link.
- `RateListItemsTable`'s paste-import still has no `FormField` /error-state UI.

New as of this edition

- **Documentation debt, now resolved rather than left open:** `DEVELOPMENT_LOG.md`'s own Eras 14 through 19 had no dedicated write-ups at the time the corresponding work shipped. A documentation catch-up pass on 2026-09-17 filled the missing stretch in as decimal-numbered entries (Era 19.5 and Era 19.6). Worth flagging here precisely because it happened inside the same window this edition covers: a real documentation gap was found and closed, not merely noted as still open.
- **Land and verify Feature #60** (the manager-permission audit and the new standalone `/profile` page) once committed. As of 2026-09-17 this work is implemented and passing every automated check and has been live-verified against the real dev database, but `CLAUDE.md`'s own "Current Work In Progress" note still records it as not yet committed.

Resolved since the Fourth Edition — no longer applicable

- **The IST/UTC date-boundary bug class, permanently closed out rather than merely patched again.** A bare `new Date("YYYY-MM-DD")` re-parse on every invoice/purchase-bill edit, a shared-object `.setHours()` mutation compounding it, and `formatDate()` / `formatDateTime()` never pinning `timeZone: "Asia/Kolkata"` at all — all fixed, with an automated pre-commit guard added so the same bug class can no longer silently regress.
 - **Purchase-bill attachments moved off a public Blob store onto a private one.** Migrated to a dedicated private-access store served only through an authenticated proxy route, including a same-day production outage the migration itself introduced, closed before the edition closed.
 - **Purchase Bill's per-line CGST/SGST footing mismatch against its own printed total.** The same rounding-mismatch bug already fixed on the invoice side was found to still exist on Purchase Bill's own item table and PDF print area, and was fixed in the same commit that removed the WAC costing engine.
 - Every critical/high npm audit finding present as of the Third Edition remains cleared, with no new critical/high findings introduced by any dependency change in this window.
-

This is the final chapter of the Science Hub Bible, Fifth Edition.

Verified against `d:\nextApps\science-hub` as of 2026-09-17. Prior editions: First (2026-06-24 era), Second (2026-07-24), Third (2026-08-17), Fourth (2026-08-31).